

• Claude on Microsoft Foundry

Anthropic Claude, Entra identity, no shared keys

Run Claude against models hosted in Microsoft Foundry —
a paved path for individual developers **and** enterprise rollouts.



Why this talk

- Claude is now a **first-class model family inside Microsoft Foundry** — same resource, same RBAC, same region story as every other Foundry model.
- Two very different consumers want it:
 - **Developers** → Claude Code on their laptop, today.
 - **Enterprises** → governed access with no API keys on devices.
- The hard part isn't "calling an API" — it's **identity, RBAC, and environment setup**.
- This deck = the field-tested patterns + the gotchas that actually bite.

The big idea: one Foundry, two tracks

	Track A — Claude Code	Track B — Claude Desktop
Client	Claude Code CLI + VS Code	Claude Desktop 1.5+
Path to Foundry	Direct	Gateway (APIM)
Auth on client	Per-user <code>az login</code> (Entra bearer)	Entra ID OIDC SSO
Keys on device	None (token cache)	None
Who it's for	Individual developers	Enterprise / IT rollout

Shared foundation: same Foundry resource · same Claude deployments · same `Foundry User` RBAC at resource scope.

Architecture at a glance

Track A — direct



Developer

-  Claude Code (CLI + VS Code)
-  `az login` session (Entra bearer)
- **Foundry**

Track B — gateway

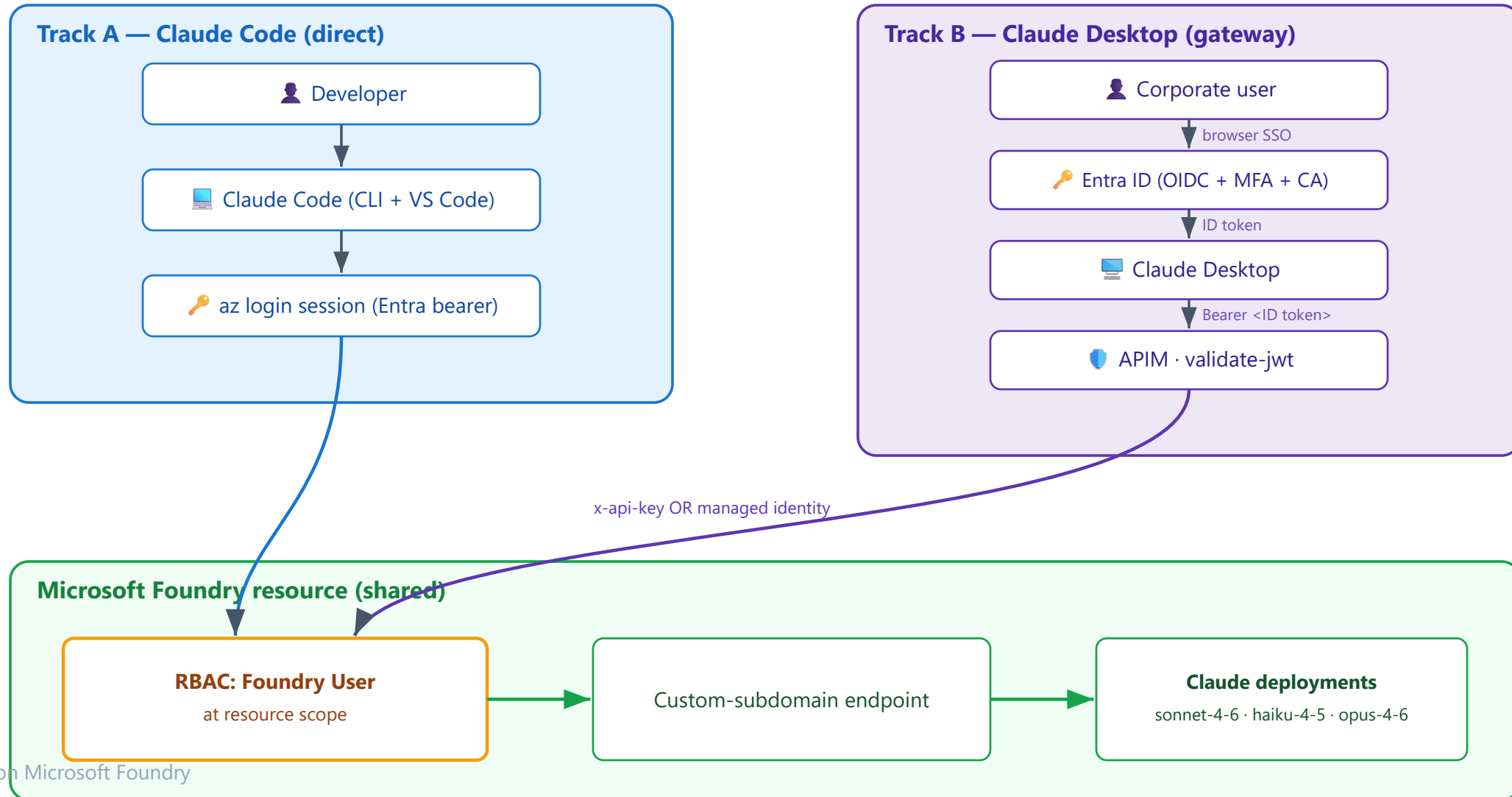


Corporate user

-  Claude Desktop
-  Entra ID (OIDC + MFA + CA)
-  APIM `validate-jwt`
- **Foundry**

Microsoft Foundry (shared): Foundry User RBAC → custom-subdomain endpoint → Claude deployments (`sonnet-4-6` · `haiku-4-5` · `opus-4-6`)

Architecture — full request flow



Shared prerequisites

- Azure subscription with a Microsoft Foundry / AI Services account.
- A Claude model deployment — minimum `claude-sonnet-4-6` (recommend also `haiku-4-5`, `opus-4-6`).
- Account in a region where Claude is offered — currently **East US 2** and **Sweden Central** (more coming).
- A custom subdomain on the Foundry account → required for AAD auth.
- `Foundry User` role assigned at the Foundry resource scope.
- Azure CLI logged into the right tenant.

⚠ Region + custom subdomain are the two setup steps people skip — and both block auth entirely.

RBAC: get this one right

- Assign **Foundry User** (role ID `53ca6127-db72-4b80-b1b0-d745d6d5456d` , formerly *Azure AI User*).
- Scope it **at the Foundry resource**, not the subscription of a different sub.
- Assign it to whichever **identity actually calls Foundry**:
 - Track A → the **developer user**.
 - Track B → the **APIM managed identity**.

⚠ Roles starting with **Cognitive Services *** do **not** apply to Foundry. Old guidance to also add *Cognitive Services User* is **obsolete** — skip it.

Track A

Claude Code — direct to Foundry

For individual developers, on their own machine, today.

Track A — how it works

1. Developer runs `az login` → Entra **bearer token** lands in the local token cache.
2. Env vars tell Claude Code to use **Foundry** as the provider.
3. Claude Code calls the Foundry endpoint **directly** with the bearer token (audience `https://ai.azure.com`).
4. Foundry checks **Foundry User** RBAC → routes to the Claude deployment.
 - No API keys anywhere. No gateway. No shared secrets.
 - Token lifecycle = the developer's normal `az login` session.

Track A — setup in 6 lines

```
az login --tenant <foundry-tenant>
export CLAUDE_CODE_USE_FOUNDRY=1
export ANTHROPIC_FOUNDRY_RESOURCE=<foundry-account>    # account NAME, not URL

claude          # then type /status
# expect: "API provider: Microsoft Foundry"
```

- Deployments are **auto-discovered by name** inside that resource.
- Only override if your deployment names differ from defaults:

```
ANTHROPIC_DEFAULT_SONNET_MODEL · ANTHROPIC_DEFAULT_HAIKU_MODEL ·
ANTHROPIC_DEFAULT_OPUS_MODEL
```

⚠ `ANTHROPIC_FOUNDRY_RESOURCE` is the **account name**, not an endpoint URL.

Track A — or just run the scripts

```
# Windows
./scripts/claude-code/setup-foundry.ps1 -ResourceName <foundry-account> -TenantId <tenant>
./scripts/claude-code/verify-setup.ps1
```

```
# macOS / Linux / WSL
./scripts/claude-code/setup-foundry.sh <foundry-account> <tenant>
./scripts/claude-code/verify-setup.sh
```

- `setup-foundry` → sets env vars + logs you in.
- `verify-setup` → confirms tenant, RBAC, endpoint, and provider before you start.

Track A — Windows reality check

⚠ Claude Code CLI needs a **POSIX shell**. Native PowerShell / `cmd.exe` can't run `claude` directly.

- Use **Git Bash** or **WSL2** to launch `claude`.
- PowerShell *can* `az login`, set env vars, and launch `code .` / `bash` — it just can't run the CLI itself.
- Env vars **only live in the shell that set them** — launch VS Code from that same shell, or use `setx` / `~/.bashrc`.
- **Reload Window** does NOT re-read parent env vars → fully quit and relaunch VS Code.

Track A — VS Code extension settings

- `claudeCode.environmentVariables` must be an array of `{name, value}` objects.
 - MS Learn shows the object form → extension **silently falls back to Anthropic**.
- `claudeCode.disableLoginPrompt: true`
 - Foundry users auth via `az login`; the Anthropic sign-in prompt is misleading and **pollutes the session** with a public token.
- `claudeCode.preferredLocation: "panel"` — preference only.

⚠ Wrong shape for `environmentVariables` = silent fallback to public Anthropic. No error, just wrong provider.

Track B

Claude Desktop — gateway with Entra SSO

For enterprises: no Foundry key on any laptop, every request carries the user's identity.

Track B — how it works

1. User signs in via **Entra ID OIDC** in the browser (MFA + Conditional Access apply).
2. Claude Desktop receives an **ID token** and sends it as **Bearer** to **APIM**.
3. APIM **validate-jwt** inbound policy **verifies** the token (issuer, audience, claims).
4. APIM forwards to Foundry using its **managed identity** (or key) — the client never sees it.
5. Foundry checks **Foundry User** RBAC on the **APIM identity** → Claude deployment.
 - Foundry credential **never leaves Azure**. Full Conditional Access + MFA enforcement.

Track B — setup at a glance

```
# 1. Fill in .env
Copy-Item .env.example .env ; notepad .env

# 2. Log in to the correct tenant
az login --tenant <your-tenant-id>

# 3. (Skip if you already have APIM)
./scripts/claude-desktop/create-apim.ps1

# 4. Register the Entra app + push APIM named values
./scripts/claude-desktop/register-claude-entra-app.ps1
```

Then wire up the APIM API, the `validate-jwt` inbound policy, and Claude Desktop's Gateway SSO settings.

Track B — the platform-type gotcha

⚠ Register the Entra app under "Mobile and desktop applications", not "Web".

- Wrong platform → `Token exchange failed (HTTP 401)` in Claude Desktop.
- The redirect URI and token-exchange flow differ between the two platform types.
- Double-check the **app registration** → **Authentication** blade before debugging anything else.

Troubleshooting — top failure modes

Symptom	Most likely cause
<code>/status</code> shows API provider: Anthropic	Env vars not inherited by launching process (A)
401 / 403 from Foundry	<code>Foundry User</code> not at resource scope
<code>baseUrl</code> and <code>resource</code> are mutually exclusive	Both <code>ANTHROPIC_BASE_URL</code> and <code>ANTHROPIC_FOUNDRY_RESOURCE</code> set (A)
Unable to get authority for <code>/<guid></code>	<code>az login</code> on the wrong tenant
Token exchange failed (HTTP 401)	Entra app registered as Web not desktop (B)
APIM 200 but Foundry 404	API-version / path mismatch on <code>/anthropic</code> backend (B)

Full matrix: `docs/troubleshooting.md`.

Decision guide — which track?

Choose Track A if...

- Individual / small team of devs
- Comfortable with `az login`
- Want Claude Code in the terminal + VS Code
- No need to broker identity through a gateway

Choose Track B if...

- Enterprise / IT-governed rollout
- Foundry key must never touch laptops
- Need MFA + Conditional Access on every call
- Using Claude **Desktop** as the client

You can run **both** — same Foundry, same deployments, same RBAC model.

Resources & enablement

- Setup walkthroughs — `docs/claude-code/setup-guide.md` , `docs/claude-desktop/blog.md`
- Architecture deep-dive — `docs/claude-code/architecture.md`
- Scripts — `scripts/claude-code/` , `scripts/claude-desktop/`
- Sample config — `examples/claude-code/vscode-settings.sample.json` , `CLAUDE.md`
- Field enablement — `enablement/cheat-sheet.md` , `enablement/scenarios.md`
- Troubleshooting matrix — `docs/troubleshooting.md`

Repo: `lindazhang2000/claude-on-foundry`

• Key takeaways

1. One Foundry, two tracks — direct (devs) and gateway (enterprise).
2. Identity is the work — Foundry User at resource scope, right tenant, custom subdomain.
3. Know the gotchas — env-var inheritance, role naming, platform type, `/status` check.

Questions?

